

REAL-TIME SIEM DASHBOARD USING PYTHON

Cyber Security Project Report

Author: Prkhar Sharma

Domain: Cyber Security

Technologies: Python, Dash, Plotly, SQLite, Pandas

Version: 1.0

Date: May 2026

Abstract

This project presents the implementation of a lightweight Security Information and Event Management (SIEM) platform developed using Python. The system monitors authentication logs in real time, parses and classifies events, stores them in SQLite, and visualizes activity through a SOC-style dashboard built using Dash and Plotly.

The platform detects brute-force attacks and invalid-user reconnaissance attempts using sliding-window threshold correlation. Automated Slack and email alerts are generated whenever suspicious activity exceeds configured thresholds. The project demonstrates practical cybersecurity monitoring concepts including log analysis, detection engineering, alert correlation, and dashboard visualization.

Introduction

Modern organizations require continuous monitoring systems capable of identifying suspicious behavior across authentication logs and system events. SIEM platforms provide centralized visibility into security events and help analysts detect threats in real time.

This project was developed to simulate the core functionality of enterprise SIEM tools using open-source Python technologies. The solution focuses on real-time log ingestion, event correlation, alert generation, and security visualization.

Objectives

- Develop a real-time log monitoring system using Python
- Detect brute-force login attacks
- Detect invalid-user reconnaissance attempts
- Generate automated Slack and email alerts
- Store security events inside SQLite
- Visualize attack activity using a SOC-style dashboard
- Demonstrate cybersecurity monitoring and automation concepts

System Architecture

The system consists of five primary components:

1. Log Parser
Continuously monitors log events and extracts structured information using regex.
2. Detection Engine
Implements sliding-window threshold logic for attack detection.
3. SQLite Database
Stores logs and alerts for persistence and analytics.
4. Dashboard
Displays KPIs, timelines, attacker IPs, alert feeds, and live logs.
5. Alerting System
Sends notifications using Slack webhooks and Gmail SMTP.

Implementation Details

The parser reads incoming authentication logs and extracts:

- Timestamp
- Host
- Program
- Message
- Source IP

Events are tagged into categories such as:

- failed_login
- invalid_user
- successful_login
- cron

- sudo

The detection engine tracks recent events using a sliding time window. If failed login attempts exceed configured thresholds within a short duration, alerts are generated automatically.

The dashboard refreshes every 5 seconds and visualizes:

- Event timelines
- Event-type distribution
- Top attacker IPs
- Alert feed
- Live log tail

Testing and Validation

The SIEM platform was tested by simulating SSH brute-force attacks and invalid-user probes using synthetic log generation commands.

Example brute-force simulation:

```
for i in {1..10}; do echo "Failed password" >> test.log; done
```

Expected Result:

- Dashboard updates in real time
- Alerts generated after threshold breach
- Slack and email notifications sent successfully

The system successfully detected attacks and updated the dashboard dynamically.

Challenges Faced

- VMware file monitoring inconsistencies
- Plotly layout conflicts in dashboard rendering
- Slack webhook integration debugging
- Gmail SMTP authentication setup
- Real-time polling and dashboard refresh optimization

Future Improvements

- Machine learning anomaly detection
- Elasticsearch integration
- Docker deployment

- Multi-host monitoring
- GeoIP visualization
- Threat intelligence feed integration
- Role-based dashboard authentication

Conclusion

The project successfully demonstrates the implementation of a lightweight SIEM platform using Python and open-source tools. It provides practical exposure to security monitoring, event correlation, dashboard visualization, and cybersecurity automation concepts.

The system simulates many features commonly found in enterprise SOC environments and serves as a strong cybersecurity portfolio project.

Security Skills Demonstrated

Skill Area	Evidence in Project
Log Analysis	Real-time parsing and event tagging
Threat Detection	Sliding-window brute-force detection
Incident Response	Alert feed and event correlation
Automation	Python-based alerting and monitoring
Dashboarding	SOC-style real-time dashboard

References

- Python Documentation — <https://docs.python.org>
- Dash Documentation — <https://dash.plotly.com>
- Plotly Documentation — <https://plotly.com/python>
- SQLite Documentation — <https://sqlite.org>
- Slack API Documentation — <https://api.slack.com>